

79. Word Search Medium

Acceptance: 46.6% | [LeetCode](#)

Given an $m \times n$ grid of characters `board` and a string `word`, return `true` if `word` exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

```
**Input:** board = [
  ["A","B","C","E"],
  ["S","F","C","S"],
  ["A","D","E","E"]
], word = "ABCCED"
**Output:** true
```

Example 2:

```
**Input:** board = [
  ["A","B","C","E"],
  ["S","F","C","S"],
  ["A","D","E","E"]
], word = "SEE"
**Output:** true
```

Example 3:

```
**Input:** board = [
  ["A","B","C","E"],
  ["S","F","C","S"],
  ["A","D","E","E"]
], word = "ABCB"
**Output:** false
```

Constraints:

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 6`
- `1 <= word.length <= 15`
- `board` and `word` consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger `board` ?

Claude Opus Solution

Word Search — Solution Guide

1. Intuition

The key insight is that we're looking for a **path** in a grid where consecutive cells spell out our target word. This is a classic **backtracking** problem.

Why backtracking? Because at each cell we have choices (4 directions to move), and if a choice leads to a dead end, we must "undo" it and try another. The constraint "same cell may not be used more than once" naturally suggests a **mark-and-unmark** pattern as we explore.

The core pattern: DFS + Backtracking - Try starting the word from every cell. - From each matching cell, explore all 4 neighbors recursively to match the next character. - Mark visited cells so we don't reuse them, then **unmark** on backtrack so other paths can use them.

2. Approach (Pseudocode)

```
function exist(board, word):
    for each cell (r, c) in board:
        if dfs(r, c, index=0) is True:
            return True
    return False

function dfs(r, c, index):
    # Base case: matched all characters
    if index == len(word):
        return True

    # Out of bounds OR character mismatch -> dead end
    if (r,c) out of bounds OR board[r][c] != word[index]:
        return False

    # MARK current cell as visited
    temp = board[r][c]
    board[r][c] = '#'

    # Explore 4 directions; succeed if ANY works
    found = dfs(r+1,c,index+1) OR dfs(r-1,c,index+1)
             OR dfs(r,c+1,index+1) OR dfs(r,c-1,index+1)

    # UNMARK (backtrack) to restore state
```

```
board[r][c] = temp

return found
```

3. Python Solution

```
from typing import List

class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        # Grid dimensions
        rows, cols = len(board), len(board[0])

        def dfs(r: int, c: int, index: int) -> bool:
            # SUCCESS: we've matched every character of `word`.
            # This check comes first so that even a 1-char word works.
            if index == len(word):
                return True

            # FAILURE conditions (pruning):
            # - Out of grid bounds, OR
            # - Current cell's char doesn't match the needed char.
            if (r < 0 or r >= rows or
                c < 0 or c >= cols or
                board[r][c] != word[index]):
                return False

            # MARK cell as visited by overwriting it with a sentinel.
            # Saving the original lets us restore on backtrack.
            temp = board[r][c]
            board[r][c] = '#'

            # Explore all 4 adjacent cells for the NEXT character.
            # `or` short-circuits: stop as soon as one path succeeds.
            found = (
                dfs(r + 1, c, index + 1) or # down
                dfs(r - 1, c, index + 1) or # up
                dfs(r, c + 1, index + 1) or # right
                dfs(r, c - 1, index + 1)    # left
            )

            # BACKTRACK: restore the cell so other search paths can use it.
            board[r][c] = temp

            return found

        # Try starting the word from every cell in the grid.
        for r in range(rows):
            for c in range(cols):
                if dfs(r, c, 0):
                    return True

        # No starting cell led to a full match.
        return False
```

4. Complexity Analysis

Time Complexity: $O(m \cdot n \cdot 4 \cdot 3^{(L-1)}) \approx O(m \cdot n \cdot 3^L)$

- We try starting from each of the $m \cdot n$ cells.
- From each cell, the first move has **4** directions, but every subsequent move has only **3** choices (we can't go back to the cell we just came from).
- The recursion depth is **L** (length of the word), so each search tree explores roughly 3^L paths.
- **L** is the word length.

Space Complexity: $O(L)$

- The recursion call stack goes at most **L** deep (one frame per matched character).

- We modify the board **in place** for marking (no extra visited array), so no additional $O(m \cdot n)$ space — just the implicit call stack.

5. Alternative Approaches

A. Separate visited set/matrix (instead of in-place marking)

```
visited = set()
# add (r,c) before recursing, remove after
```

- Cleaner/safer if you can't mutate the input, but costs $O(m \cdot n)$ extra space. The in-place trick is preferred for efficiency.

B. Search Pruning (the Follow-up optimization)

For larger boards, add these optimizations to fail fast:

1. **Character-frequency check:** Before searching, verify every character in `word` exists in `board` with sufficient frequency. If not, return `False` immediately.
2. **Reverse the word if its last char is rarer than its first.** Starting DFS from the rarer endpoint creates fewer search branches: `python from collections import Counter board_count = Counter(ch for row in board for ch in row) # If the last letter is rarer in the board than the first, # searching for the reversed word explores fewer paths. if board_count[word[-1]] < board_count[word[0]]: word = word[::-1]`
3. **Early frequency abort:** `python word_count = Counter(word) if any(word_count[ch] > board_count[ch] for ch in word_count): return False # impossible - not enough letters`

C. Trie-based search — Not beneficial for a *single* word, but if searching for **multiple words simultaneously** (Word Search II, LeetCode 212), building a Trie of all words and doing one DFS is far more efficient than running this algorithm per word.

Key Takeaways

- **Pattern recognition:** Grid + path-with-no-reuse → **DFS + Backtracking**.
- **The mark/unmark idiom** is the heart of backtracking — always restore state after exploring.
- **Short-circuit** or elegantly stops early when any direction succeeds.

Codex Solution

1. Intuition

This is a classic **backtracking / DFS on a grid** problem.

We need to form `word` by moving through adjacent cells, but:

- We can move only **up, down, left, right**
- We cannot reuse the same cell in one path

So from every cell that matches `word[0]`, we try to build the word character by character using DFS.

The key pattern is:

Try a choice, mark it as used, recursively continue, then undo the choice.

This is exactly **backtracking**.

We can also add pruning:

1. If `len(word) > m * n`, impossible.
2. If the board does not contain enough of some character, impossible.
3. Start searching from the rarer end of the word to reduce branching.

2. Approach

Pseudocode

```
exist(board, word):
    m = number of rows
```

```

n = number of columns

if len(word) > m * n:
    return false

count characters in board
count characters in word

for each character in word:
    if board does not have enough occurrences:
        return false

if last character of word is rarer than first character:
    reverse word

define dfs(row, col, index):
    if index == len(word):
        return true

    if row or col is out of bounds:
        return false

    if board[row][col] != word[index]:
        return false

    temporarily mark board[row][col] as visited

    search in 4 directions:
        down, up, right, left

    restore board[row][col]

    return whether any direction succeeded

for each cell in board:
    if dfs(cell, 0) returns true:
        return true

return false

```

3. Python Solution

```

from typing import List
from collections import Counter

class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        rows = len(board)
        cols = len(board[0])
        word_len = len(word)

        # -----
        # Pruning 1:
        # If the word is longer than the total number of cells,
        # it is impossible because each cell can be used at most once.
        # -----
        if word_len > rows * cols:
            return False

        # -----
        # Pruning 2:
        # Check whether the board contains enough of every character
        # needed by the word.
        # -----
        board_count = Counter()

        for r in range(rows):
            for c in range(cols):
                board_count[board[r][c]] += 1

```

```

word_count = Counter(word)

for ch, needed in word_count.items():
    if board_count[ch] < needed:
        return False

# -----
# Pruning 3:
# Start from the rarer side of the word.
#
# Example:
# If word[0] appears many times but word[-1] appears once,
# searching the reversed word may start from fewer cells.
# -----
if board_count[word[-1]] < board_count[word[0]]:
    word = word[::-1]

# Four possible movement directions:
# down, up, right, left
directions = [
    (1, 0),
    (-1, 0),
    (0, 1),
    (0, -1),
]

def dfs(row: int, col: int, index: int) -> bool:
    """
    Returns True if word[index:] can be matched starting
    from board[row][col].
    """

    # -----
    # If index reaches the word length, we have matched
    # every character successfully.
    # -----
    if index == len(word):
        return True

    # -----
    # Boundary check.
    # -----
    if row < 0 or row >= rows or col < 0 or col >= cols:
        return False

    # -----
    # Character mismatch.
    # Also handles visited cells because we mark them with '#',
    # which cannot equal a letter from the word.
    # -----
    if board[row][col] != word[index]:
        return False

    # -----
    # Mark current cell as visited.
    #
    # We save the original character so we can restore it later.
    # This avoids using an extra visited set.
    # -----
    original_char = board[row][col]
    board[row][col] = "#"

    # -----
    # Explore all four neighboring cells.
    # If any path works, we can return True.
    # -----
    for dr, dc in directions:
        next_row = row + dr
        next_col = col + dc

        if dfs(next_row, next_col, index + 1):
            # Restore before returning.
            board[row][col] = original_char

```

```

        return True

    # -----
    # Backtrack:
    # Restore the current cell before returning False.
    # -----
    board[row][col] = original_char
    return False

# -----
# Try to start the DFS from every cell.
# The DFS itself checks if the cell matches word[0].
# -----
for r in range(rows):
    for c in range(cols):
        if dfs(r, c, 0):
            return True

return False

```

4. Complexity Analysis

Let:

- m = number of rows
- n = number of columns
- L = length of `word`

Time Complexity

$O(m * n * 3^L)$

More precisely:

$O(m * n * 3^{(L - 1)})$

Explanation:

- We may start DFS from each of the $m * n$ cells.
- From the first cell, there are at most 4 choices.
- After moving to one neighbor, we cannot immediately go back to the previous cell, so each later step has at most 3 choices.
- Therefore, the DFS branching is roughly $4 * 3^{(L - 1)}$.

The pruning steps add:

$O(m * n + L)$

for character counting, which is dominated by the DFS in the worst case.

So total worst-case time is:

$O(m * n * 3^L)$

Space Complexity

$O(L)$

Explanation:

- We modify the board in-place to mark visited cells.
- The recursion depth can be at most L , the length of the word.

If we used a separate `visited` set instead, the space complexity would be:

$O(L)$

for the current path, or potentially $O(m * n)$ depending on implementation.

5. Alternative Approaches

1. DFS With a `visited` Set

Instead of modifying the board in-place, we can maintain a set of visited cells:

```
visited.add((row, col))  
...  
visited.remove((row, col))
```

This is often easier to understand, but slightly slower due to set operations.

2. Trie-Based Search

If we needed to search for **many words** in the same board, a Trie would be better.

This is the idea behind LeetCode 212: **Word Search II**.

Instead of running DFS separately for every word, we:

- Insert all words into a Trie
- DFS once from each board cell
- Stop early when the current path is not a Trie prefix

For this single-word problem, regular DFS backtracking is simpler and optimal enough.